Java Full Stack Development

3. Reactive Programming

Module Topics

- Event-Driven Architecture
- Reactive Programming
- WebClient

Event-Driven Architecture

Event-Driven Architecture

- Event-driven" is used to describe two related but distinct ideas
  - Event-driven architecture: a system-level integration style
    - How services communicate with each other
    - Concerns: topics, contracts, fan-out, decoupling
  - Event-driven programming: a code-level execution model
    - How a single program reacts to inputs
    - Concerns: handlers, callbacks, async execution
  - They reinforce each other but you can have one without the other
  - This module covers both

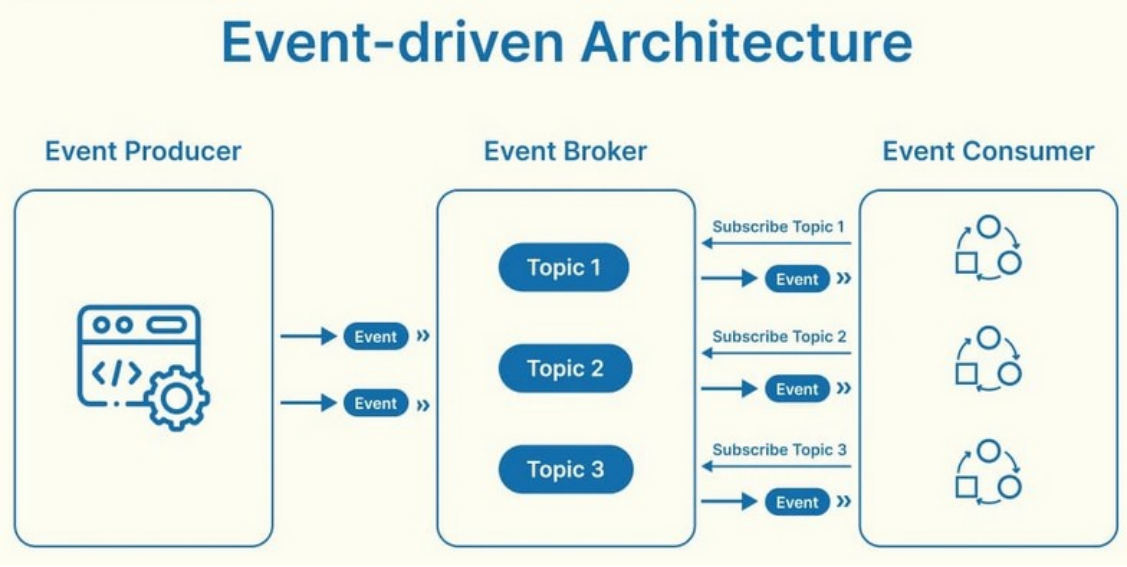


Image Credit: https://www.moontechnolabs.com/blog/software-architecture-patterns/

© 2026 by Innovation In Software Corporation

Events

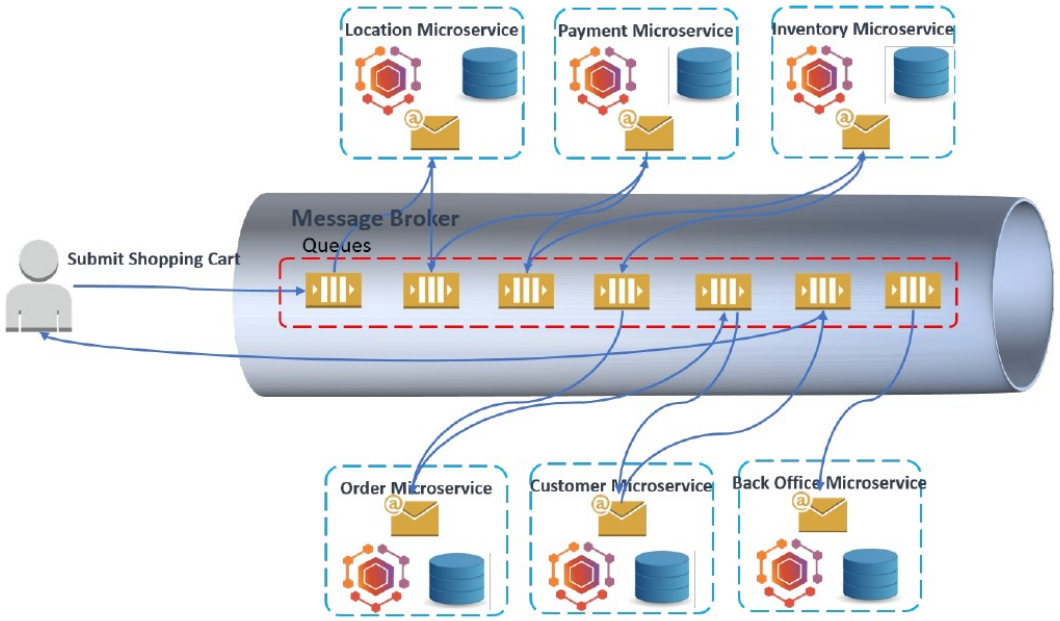
- Events
  - Unit of Integration: What services exchange
  - In a traditional microservice system: services exchange requests
    - "Service A asks Service B to do something"
    - Synchronous, point-to-point, immediate response expected
  - In an event-driven system: services exchange events
    - "Service A announces that something happened"
    - Asynchronous, broadcast, no response expected
  - The shift from "asking" to "announcing" is the heart of event-driven architecture

Aspect	Request/Response	Event-Driven
Communication	Caller asks, callee responds	Producer announces, consumers react
Coupling	Caller knows callee	Producer doesn't know consumers
Timing	Synchronous (caller waits)	Asynchronous (no one waits)
Reliability	Immediate failure if callee is down	Producer succeeds; consumer catches up later
Adding a new consumer	Modify producer to call new service	Add new consumer; no producer change
Result delivery	Returned to caller	Goes wherever consumers send it

© 2026 by Innovation In Software Corporation

Participants

- Producers, Consumers, and Decoupling
  - Producer: the service that publishes the event
    - Knows about its own domain
    - Does not know who consumes its events
  - Consumer: the service that reacts to the event
    - Knows about events it cares about
    - Does not need permission from the producer
  - The producer and consumer are decoupled through the event broker
  - Each can change, deploy, scale, or fail independently



© 2026 by Innovation In Software Corporation

Characteristics

- Event driven features
  - Asynchronous communication by default, no waiting for responses
  - Producers don't know consumers
    - Uses broadcasts, not directed calls
  - Consumers react
    - They're called by infrastructure, not by other services
  - Temporal decoupling
    - Services run on independent schedules
  - One event, many reactions
    - Fan-out is natural and free

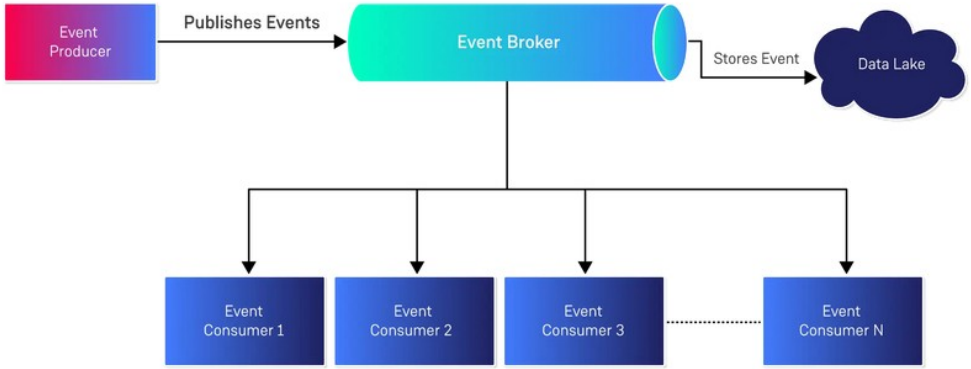


Image Credit: https://www.contino.io/insights/event-driven-architecture

© 2026 by Innovation In Software Corporation

Asynchronous by Default

- Nobody is waiting
  - The producer publishes an event and continues immediately
  - The consumer receives the event when it's ready to process
  - There is no point in the system where one service is blocked waiting for another
  - Latency between producer action and consumer reaction is variable
  - This is fundamentally different from synchronous integration

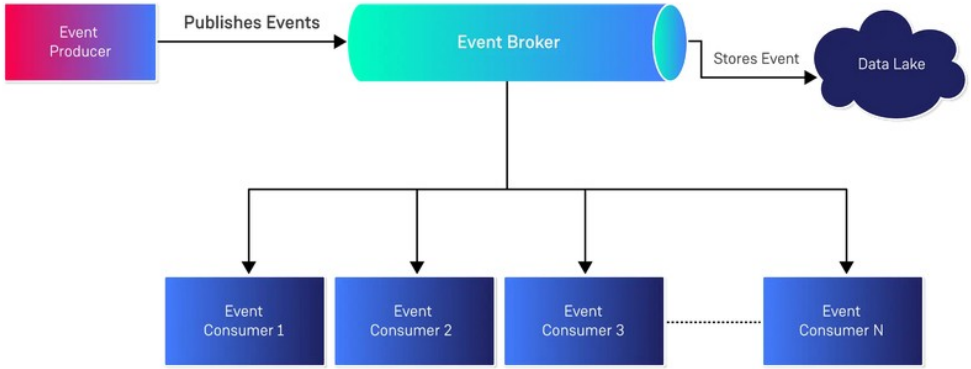


Image Credit: https://www.contino.io/insights/event-driven-architecture

© 2026 by Innovation In Software Corporation

Loose Coupling

- A producer's code never names a consumer service
- A producer's configuration never lists who receives its events
- Producers publish to topics, not to services
- New consumers can be added without producer awareness or changes
- This is the property that enables genuine architectural extensibility

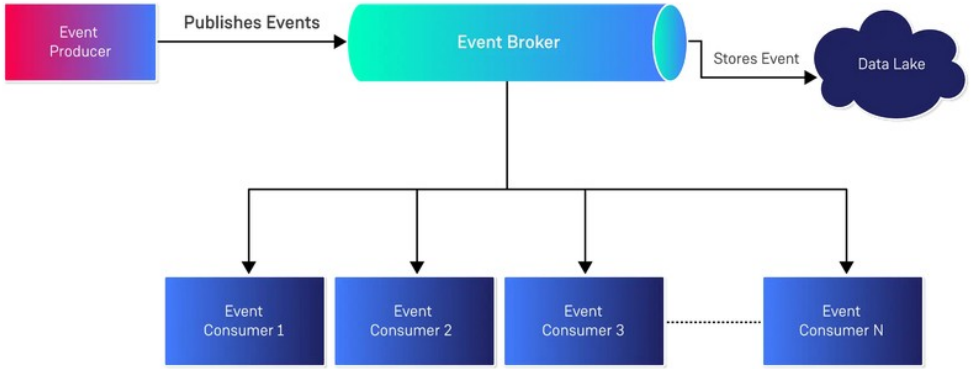


Image Credit: https://www.contino.io/insights/event-driven-architecture

© 2026 by Innovation In Software Corporation

Consumers React

- Inversion of Control
  - At the architecture Level
  - Conventional services are active
    - They decide when to call others
  - Event-driven consumers are reactive
    - They're triggered by arriving events
  - The infrastructure (broker + container) drives consumer execution
  - The consumer's code says
    - "when X arrives, do Y"
    - Not "go fetch X"

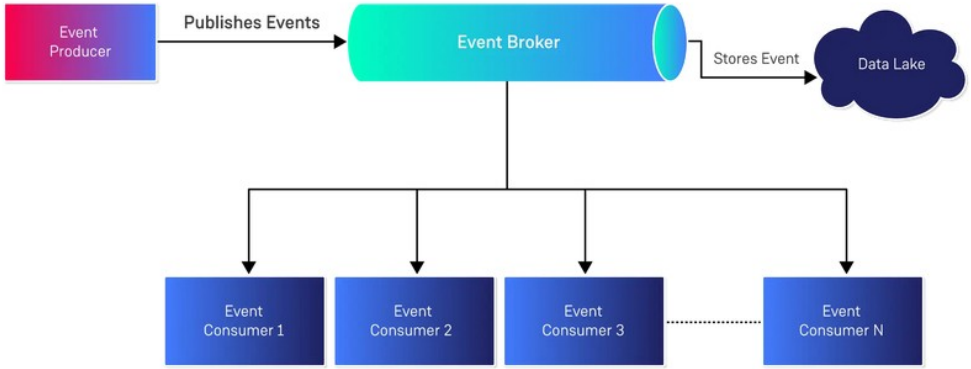


Image Credit: https://www.contino.io/insights/event-driven-architecture

© 2026 by Innovation In Software Corporation

Temporal Decoupling

- Services run non their own schedules
  - A producer can publish at 9:00 AM
  - A consumer can process at 9:01 AM, or 9:30 AM, or after a 2-hour outage
  - The event broker buffers events during gaps
  - No service needs to coordinate uptime with another
  - Maintenance windows, deployments, and failures become local concerns

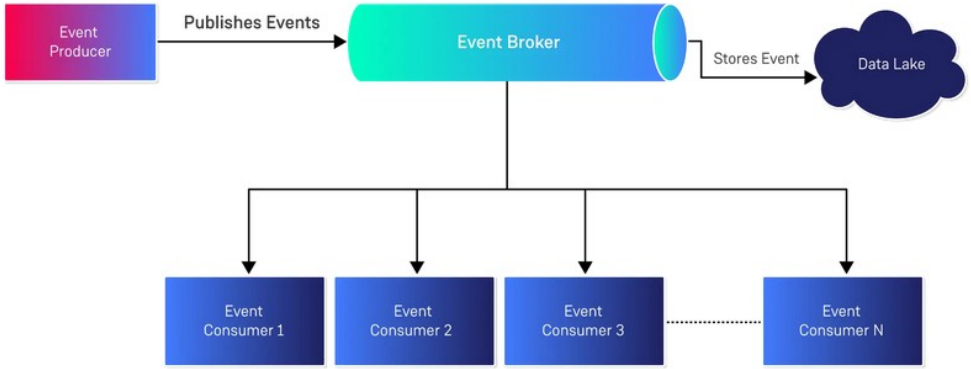


Image Credit: https://www.contino.io/insights/event-driven-architecture

© 2026 by Innovation In Software Corporation

One Event, Many Reactions

- Fan-Out as a first-class property
  - A single event can trigger reactions in many independent consumers
  - Each consumer processes the event for its own purpose
  - Adding a new consumer = adding a new reaction; no impact on others
  - Examples for OrderPlaced
    - Inventory service: reserve stock
    - Notification service: send confirmation email
    - Analytics service: update dashboards
    - Audit service: log for compliance
    - Recommendation service: update customer model

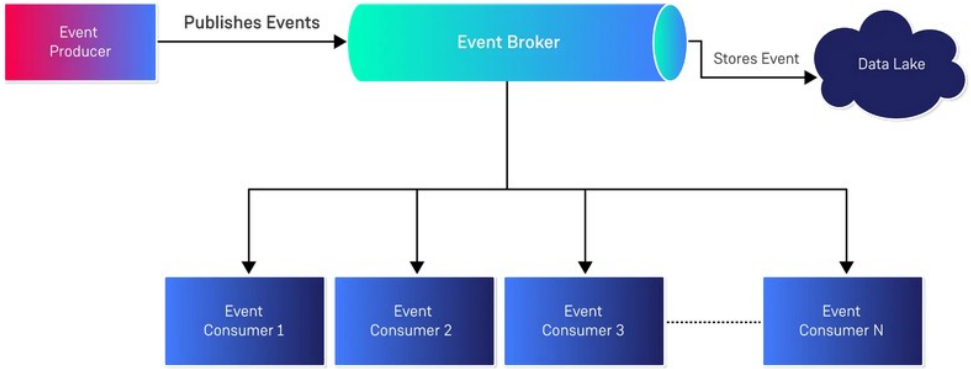


Image Credit: https://www.contino.io/insights/event-driven-architecture

© 2026 by Innovation In Software Corporation

Benefits

- Real world drivers
  - Independent deployment
    - Teams ship without coordinating with consumers
  - Resilience
    - Downstream failures don't propagate to upstream services
  - Audit trail
    - The event stream is a chronological record of what happened
  - Extensibility
    - New consumers integrate without changing existing services
  - Scalability
    - Producers and consumers scale independently
  - The architecture rewards organizational scale

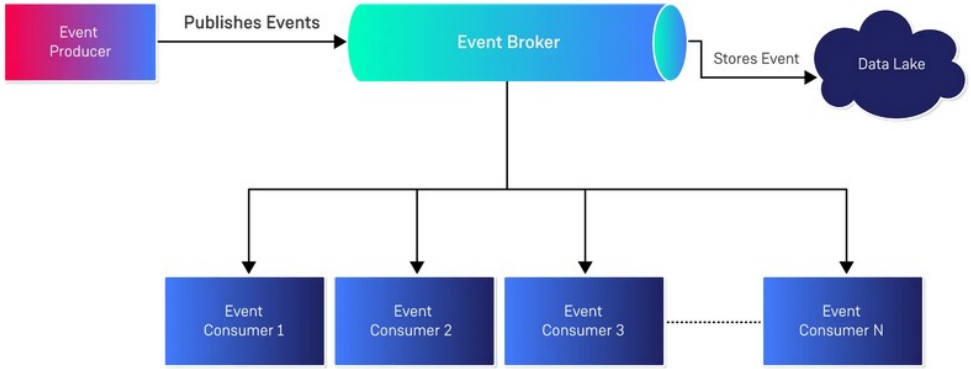


Image Credit: https://www.contino.io/insights/event-driven-architecture

© 2026 by Innovation In Software Corporation

The Costs

- There is no free lunch
  - Eventual consistency
    - Consumers see updates after the producer commits which can result in stale data
  - Harder debugging
    - No single trace from request to result
  - More moving parts
    - Broker, consumers, dead-letter handling, monitoring
  - Operational complexity
    - Schema evolution, lag monitoring, replay procedures
  - Skill investment
    - Teams need to think differently about state and time

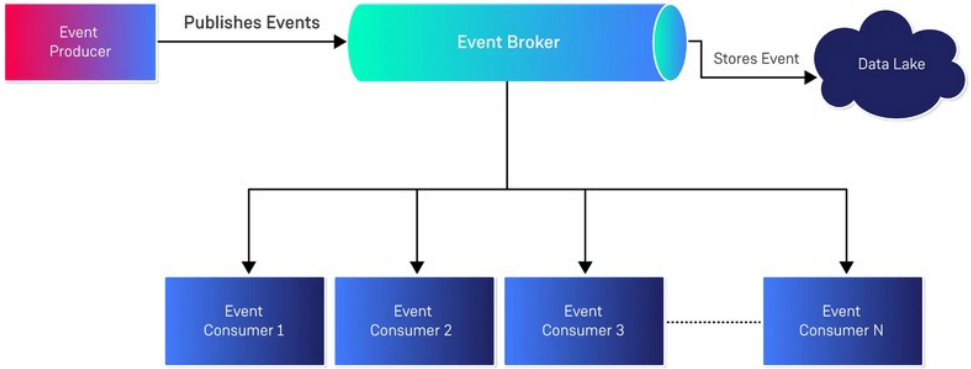


Image Credit: https://www.contino.io/insights/event-driven-architecture

© 2026 by Innovation In Software Corporation

Kafka

- Kafka as the event backbone
  - The architectural pattern needs an event broker
    - Something that receives, stores, and delivers events
  - Kafka is the dominant choice for this role at scale because:
    - It's durable (events survive consumer outages)
    - It supports many independent consumers per topic
    - It scales horizontally to large volumes
    - It allows replay (consumers can re-read history)
    - The producer talks to Kafka, not to consumers
    - The consumer talks to Kafka, not to producers
  - Kafka is the integration boundary between teams

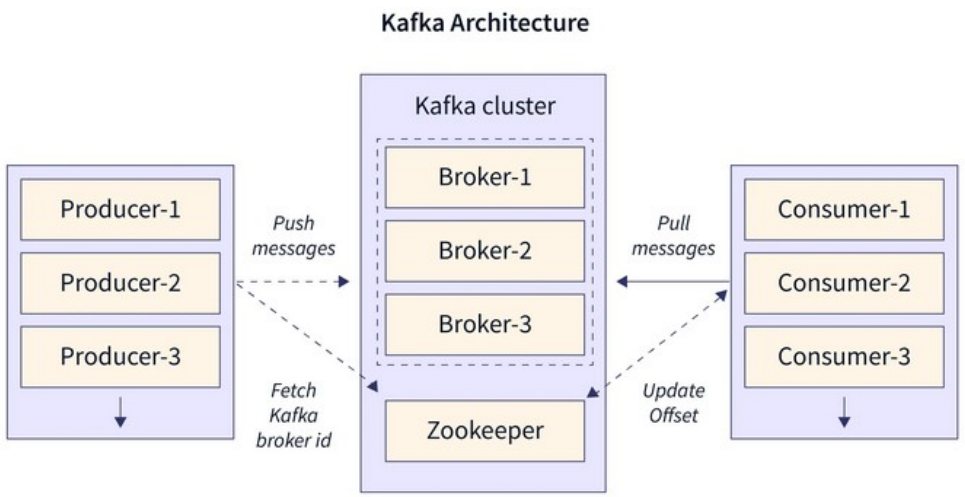


Image Credit: https://www.scaler.com/topics/kafka-architecture/

© 2026 by Innovation In Software Corporation

Log Model

- Why Kafka?
  - Kafka's log model gives every architectural property a concrete mechanism:
    - Durability: events persisted to disk, replicated across brokers
    - Fan-out: consumer groups read independently from the same log
    - Temporal decoupling: events stay available regardless of consumer state
    - Replay: consumers can reset their position and re-read
    - Scalability: partitioning distributes load horizontally
  - The architecture and the infrastructure reinforce each other

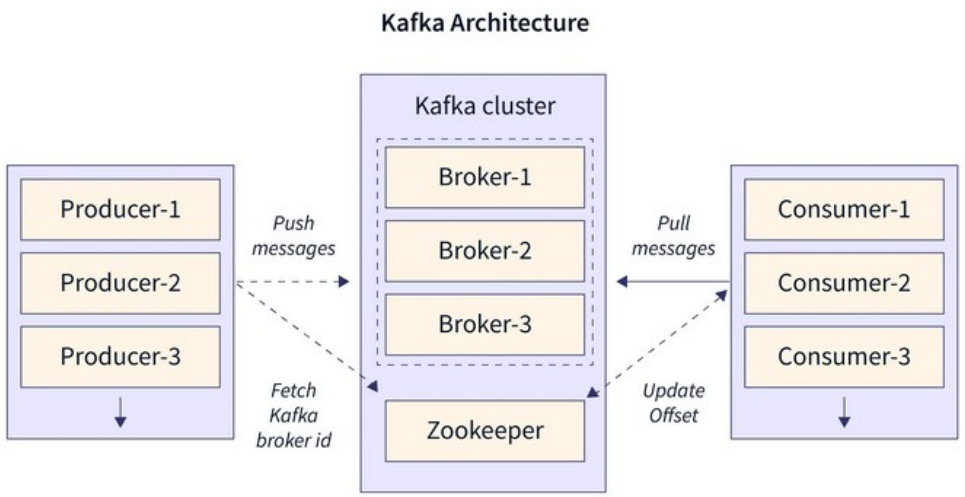


Image Credit: https://www.scaler.com/topics/kafka-architecture/

© 2026 by Innovation In Software Corporation

The Costs

- There is no free lunch
  - Eventual consistency
    - Consumers see updates after the producer commits which can result in stale data
  - Harder debugging
    - No single trace from request to result
  - More moving parts
    - Broker, consumers, dead-letter handling, monitoring
  - Operational complexity
    - Schema evolution, lag monitoring, replay procedures
  - Skill investment
    - Teams need to think differently about state and time

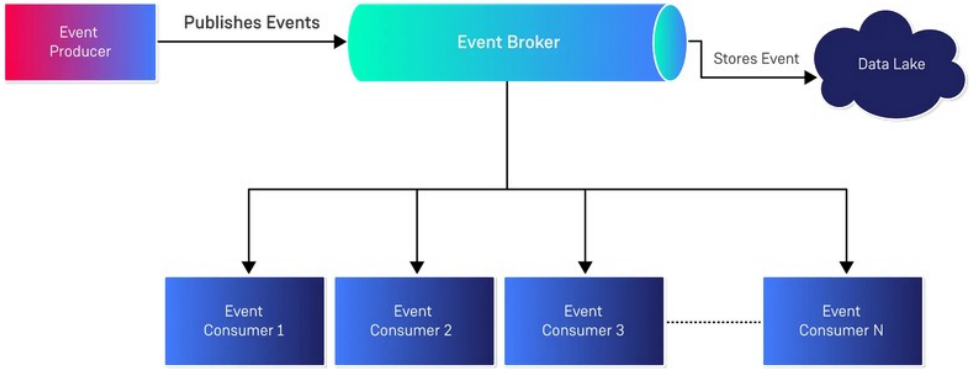


Image Credit: https://www.contino.io/insights/event-driven-architecture

© 2026 by Innovation In Software Corporation

Reactive Programming

© 2026 by Innovation In Software Corporation

Mental Shift

- From calling to reacting
  - Conventional programming
    - "I do something when I decide to"
    - Code controls when work happens
  - Procedural flow from top to bottom
  - Event-driven programming
    - "I do something when something happens"
    - Code declares reactions but the runtime triggers them
    - Flow is determined by events arriving, not by code structure
  - The shift requires thinking in handlers, not procedures

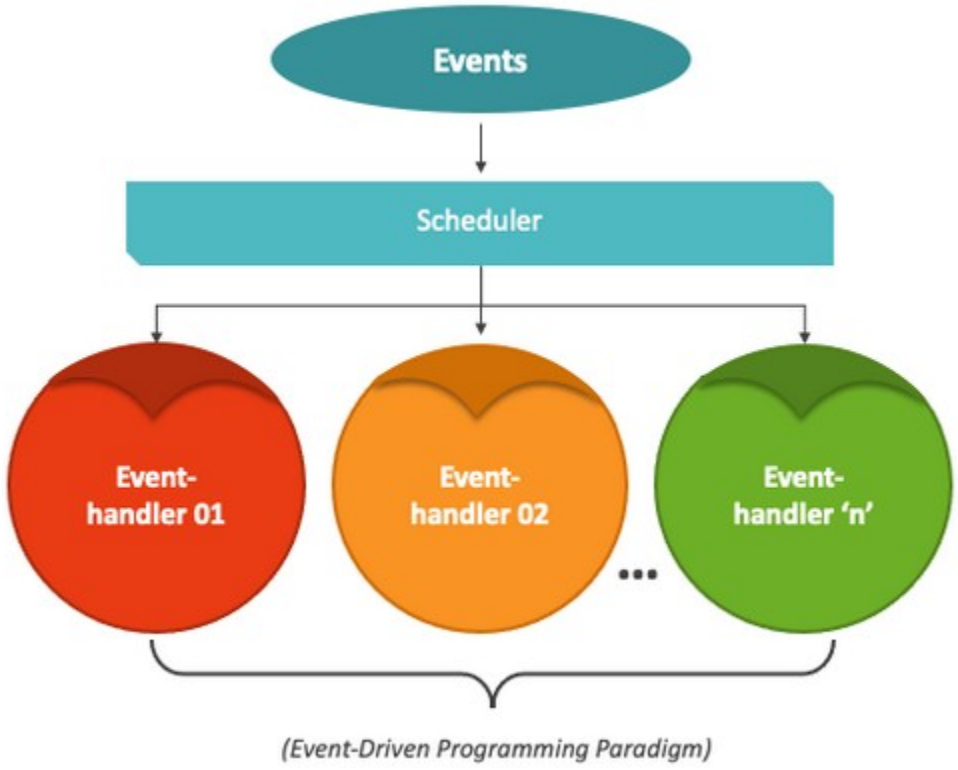


Image Credit: https://www.collidu.com/presentation-event-driven-programming

© 2026 by Innovation In Software Corporation

Mental Shift

- Inversion of control
  - In procedural code
    - Your code calls libraries
  - In event-driven code
    - Libraries call your code
  - This is inversion of control
    - The framework drives execution
    - You declare what to do
    - The framework decides when
  - Familiar from Spring itself
    - Containers and dependency injection

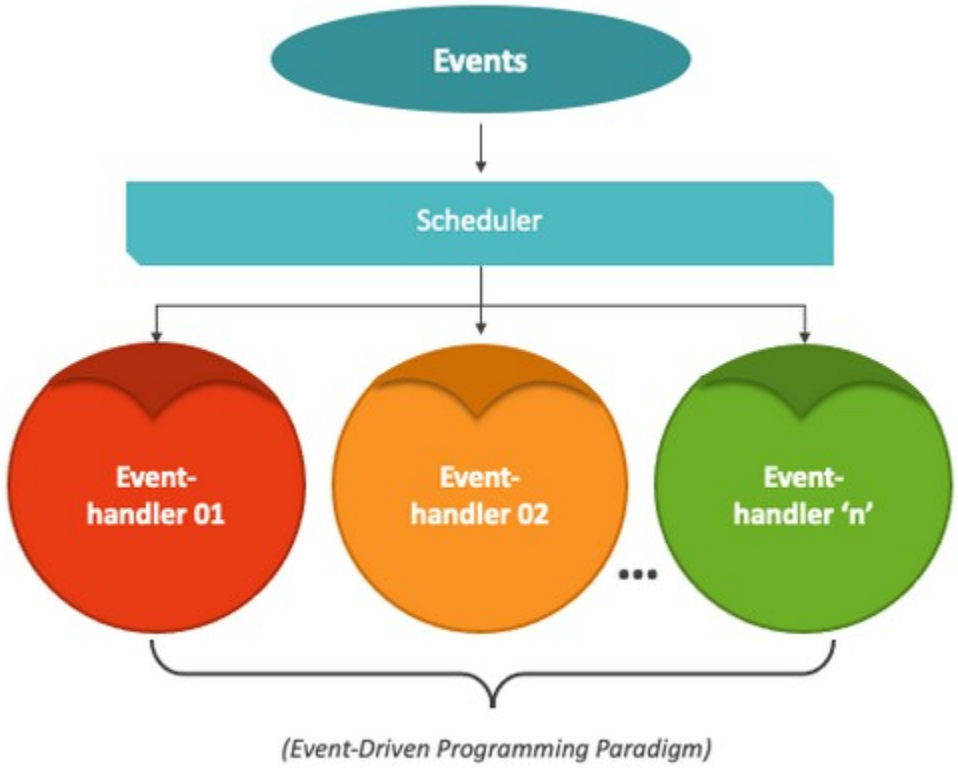


Image Credit: https://www.collidu.com/presentation-event-driven-programming

© 2026 by Innovation In Software Corporation

Mental Shift

- Nothing new, seen in
  - UI event handlers
    - button.onClick(handler) runs when the user clicks
  - Database triggers
    - SQL code runs when a row changes
  - Observer pattern
    - Subjects notify observers of state changes
  - Operating system signals
    - Handlers run when signals arrive
  - Spring's ApplicationEvent
    - beans react to events published in the context

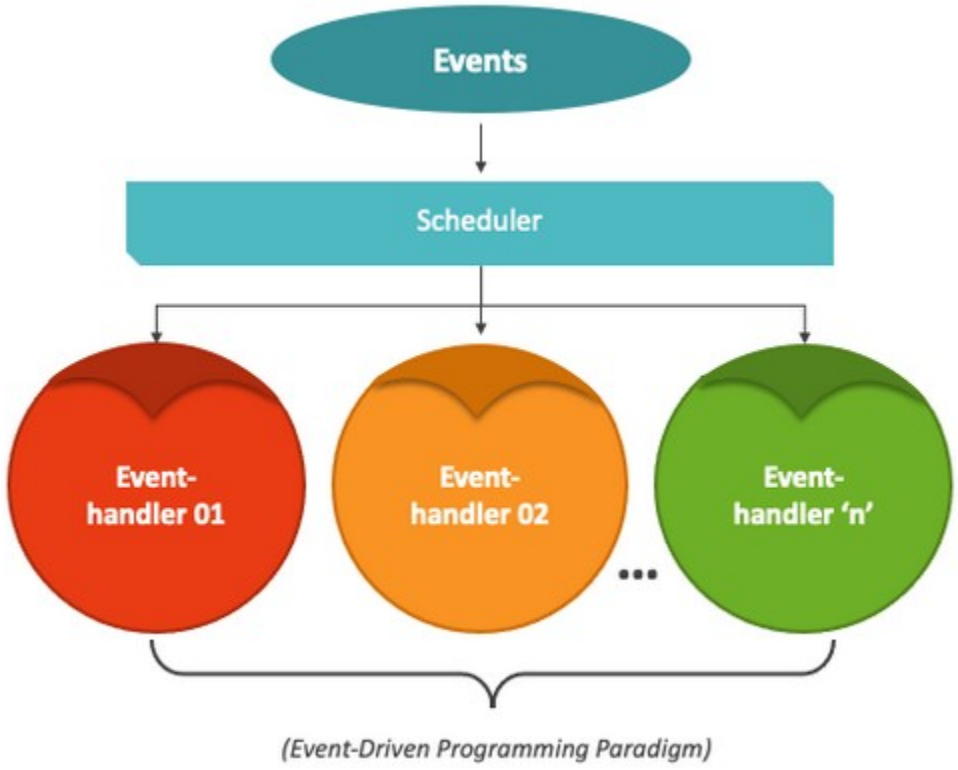


Image Credit: https://www.collidu.com/presentation-event-driven-programming

© 2026 by Innovation In Software Corporation

Core Vocabulary

- Terminology
  - Event
    - A notification that something happened
    - Past tense, factual, immutable
    - Examples: OrderPlaced, UserLoggedIn, FileUploaded
  - Handler
    - Code that runs in response to an event
    - One handler per event type, typically
    - Focused, small, single-responsibility
  - Dispatcher
    - The mechanism that routes events to handlers
    - Often called the event loop, broker, or container
  - Callback
    - A function registered to run later, when a condition is met

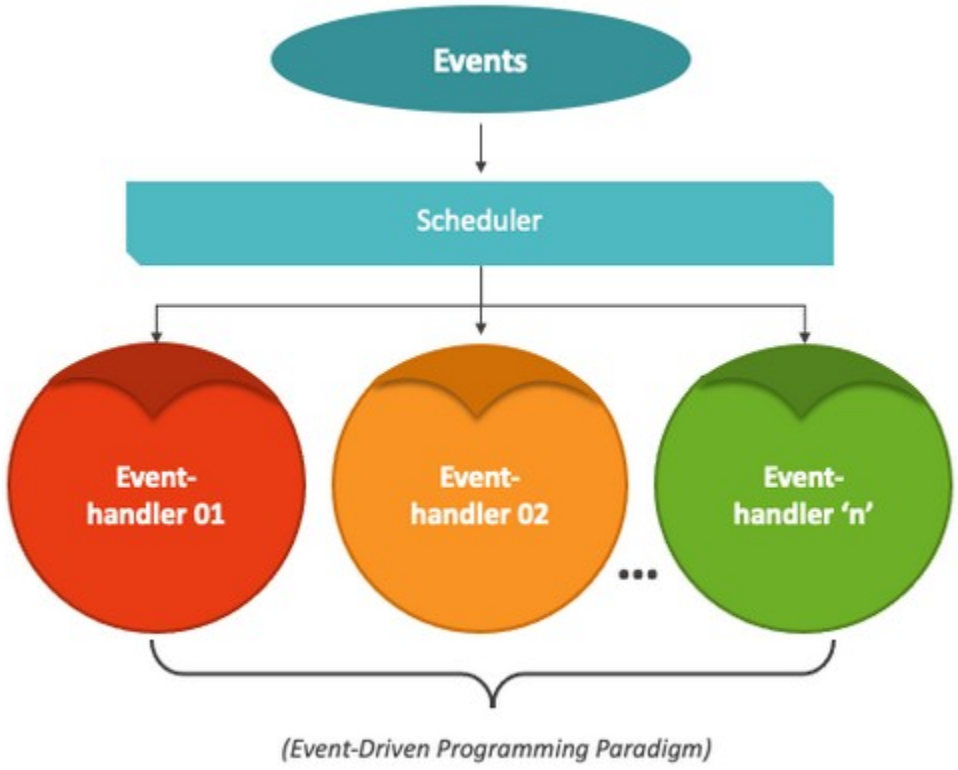


Image Credit: https://www.collidu.com/presentation-event-driven-programming

© 2026 by Innovation In Software Corporation

Events Are Facts

- Good event names
  - Are past tense and factual
  - OrderPlaced
  - PaymentReceived
  - ProfileUpdated
- Bad event names
  - Are commands or queries in disguise
    - PlaceOrder (that's a command)
    - GetUserProfile (that's a query)
    - ProcessPayment (that's a command)
- Naming discipline shapes how you think about the system

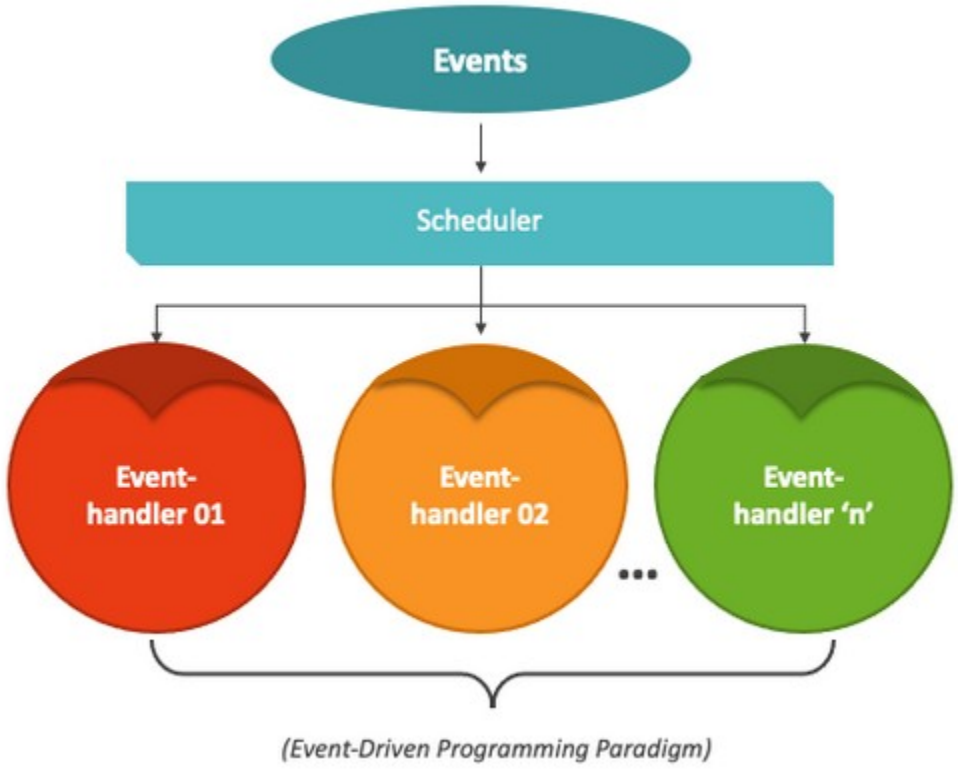


Image Credit: https://www.collidu.com/presentation-event-driven-programming

© 2026 by Innovation In Software Corporation

Synchrony

- Two ways to wait for a result
- Synchronous
  - The calling thread blocks until the result is ready
- Asynchronous
  - The calling thread continues; the result arrives later
- The difference is what the calling thread does while waiting

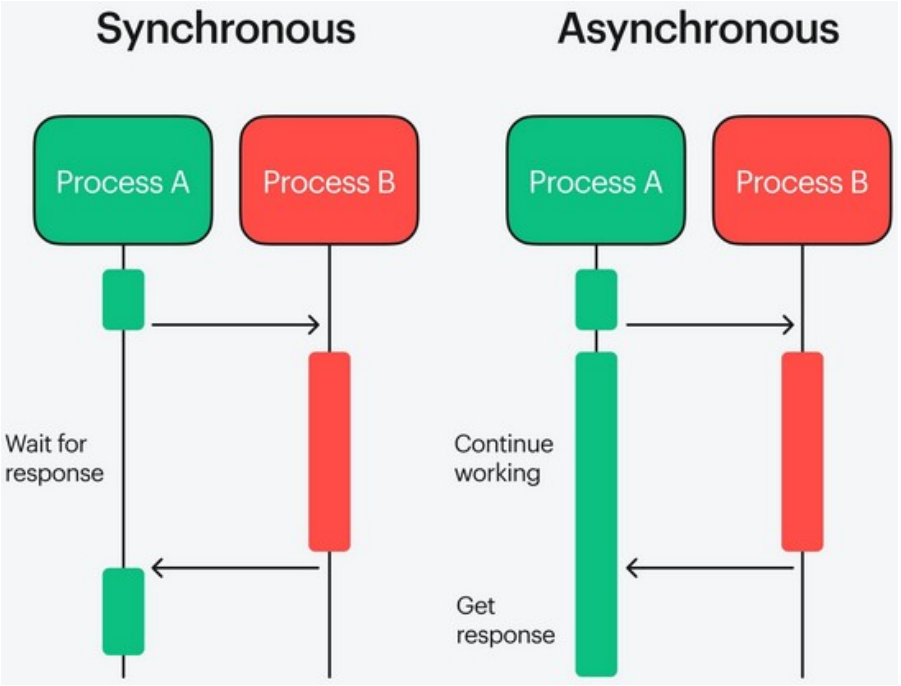


Image Credit: https://www.ramotion.com/blog/synchronous-vs-asynchronous-programming/

© 2026 by Innovation In Software Corporation

Why Async Matters

- Threads are expensive
  - A blocked thread holds:
    - A stack (typically 256 KB to 1 MB of memory)
    - A connection to the resource it's waiting on
    - A slot in the OS thread table
  - A typical JVM might support a few thousand threads
    - A typical service might need to handle tens of thousands of concurrent operations
  - Asynchronous code lets one thread handle many concurrent operations
  - The bottleneck shifts from threads to actual work

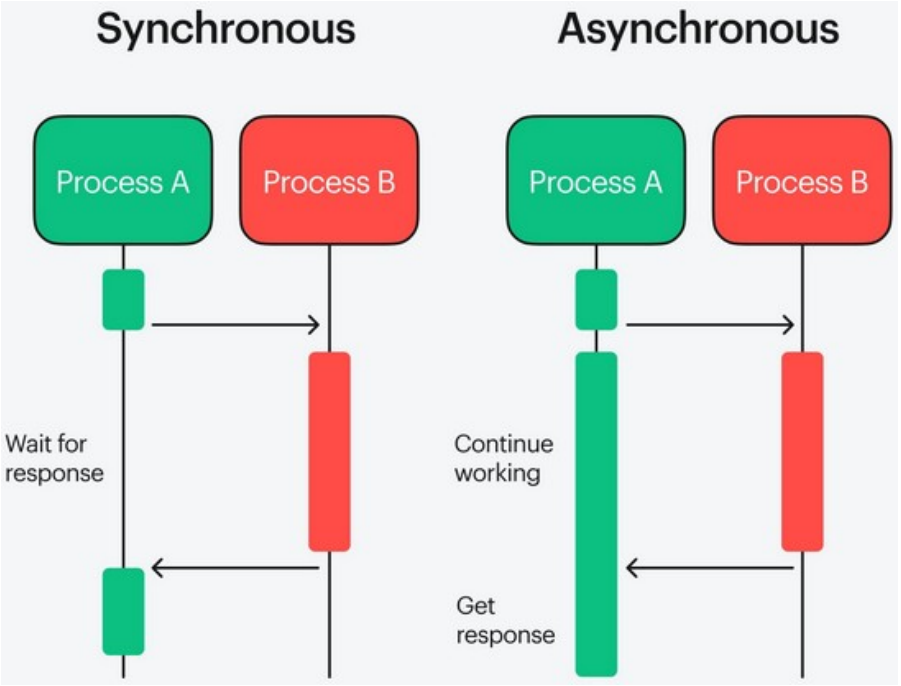


Image Credit: https://www.ramotion.com/blog/synchronous-vs-asynchronous-programming/

© 2026 by Innovation In Software Corporation

The Trade-off

- Async code is harder to read
  - The synchronous version on the left is a sequence
    - Do this, then this, then this, then return. Easy to follow, easy to debug
  - The async version on the right is a chain of nested callbacks
    - Do this, when it completes do that, when that completes do the next thing
    - Each step is a separate function
    - The flow of execution is still there, but it's expressed differently
    - This nested-callback style was historically called "callback hell" and was genuinely painful to work with.
    - Abstractions that make async code more linear-looking

- Synchronous code reads top to bottom:

```
String user = fetchUser(id);
List<Order> orders = fetchOrders(user);
Summary summary = summarize(orders);
return summary;
```

- Asynchronous code is structured as **continuations**:

```
fetchUser(id, user ->
  fetchOrders(user, orders ->
    summarize(orders, summary ->
      respond(summary))));
```

© 2026 by Innovation In Software Corporation

Async Tools in Java

- Three ways to express Async
  - All three are tools for expressing the same underlying idea: work that completes later
  - They differ in ergonomics and composability
  - Modern Spring code mixes them depending on context
    - Reactive types Mono and Flux from Project Reactor are a more recent addition
  - They generalize the idea
    - Mono<T> is a future of zero or one value
    - Flux<T> is a stream of zero to many future values
    - They come with a rich set of operators for composition
    - Spring WebClient uses these natively.

Tool	Style	When You'll See It
Plain callbacks	Pass a function to be called later	Older APIs, JavaScript-influenced code
`CompletableFuture<T>`	Composable future result	Modern Java; Spring's `KafkaTemplate.send()`
Reactive types (`Mono` **,** `Flux` **)**	Streams of future values with operators	Spring WebClient, WebFlux, R2DBC

© 2026 by Innovation In Software Corporation

CompletableFuture

- Modern Java Async type
  - thenCompose
    - Chain another async operation
  - thenApply
    - Transform the result synchronously
  - thenAccept
    - Consume the result without returning anything
  - exceptionally
    - Handle errors anywhere in the chain
  - The chain reads almost like sequential code, but each step is async

```
CompletableFuture<User> userFuture = fetchUserAsync(id);

userFuture
    .thenCompose(user -> fetchOrdersAsync(user))
    .thenApply(orders -> summarize(orders))
    .thenAccept(summary -> log.info("Summary: {}", summary))
    .exceptionally(ex -> {
        log.error("Failed", ex);
        return null;
    });
```

© 2026 by Innovation In Software Corporation

Reactive Types

- Mono and Flux at a glance
  - Mono<T>
    - A future of zero or one value
    - Like CompletableFuture<Optional<T>>, but with reactive operators
  - Flux<T>
    - A stream of zero to many future values
    - Like an asynchronous Stream<T> that emits over time
  - Both come with rich operators: map, flatMap, filter, reduce, etc.
  - Nothing happens until you subscribe
    - This is the key difference from CompletableFuture

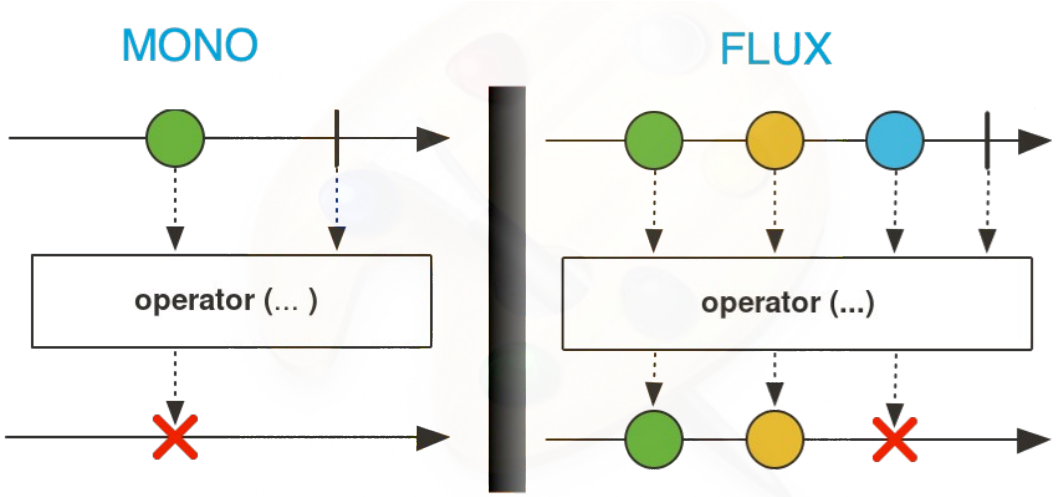


Image Credit: https://medium.com/ralvin/project-reactor-um-pouco-sobre-mono-e-flux-6f5dbf09ac49

© 2026 by Innovation In Software Corporation

Programming Model

- Architecture meets programming model
  - Event-driven systems generate events at unpredictable times
  - Handlers are naturally structured as callbacks
  - I/O-bound workloads (network, disk) benefit dramatically from async
  - The programming model mirrors the architecture:
    - Architecture: services react to events from other services
    - Code: methods react to events from infrastructure
- Same conceptual shape at two different scales

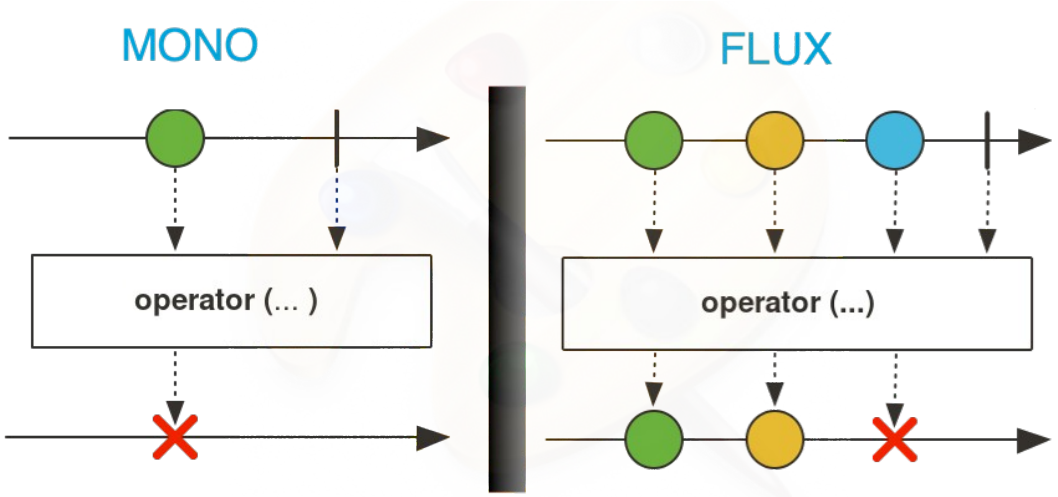


Image Credit: https://medium.com/ralvin/project-reactor-um-pouco-sobre-mono-e-flux-6f5dbf09ac49

© 2026 by Innovation In Software Corporation

Kafka Consumers

© 2026 by Innovation In Software Corporation

@KafkaListener

- @KafkaListener method
  - This is a handler in the event-driven programming sense
  - Spring Kafka is the dispatcher
  - The OrderPlacedEvent arrival is the trigger
  - Your code declares what to do; Spring decides when
  - It is inversion of control applied to message consumption

```
@KafkaListener(topics = "orders.placed")
public void handleOrderPlaced(OrderPlacedEvent event) {
    process(event);
}
```

© 2026 by Innovation In Software Corporation

Push vs. Pull

- The container drives execution
  - Your code does not contain a loop that asks "are there new events?"
  - The container runs that loop on your behalf
  - When events arrive, the container invokes your handler with each one
  - This is the same pattern as:
    - A button click handler being invoked by the UI framework
    - An @EventListener method being invoked by Spring's event publisher
    - A REST controller method being invoked by the HTTP request

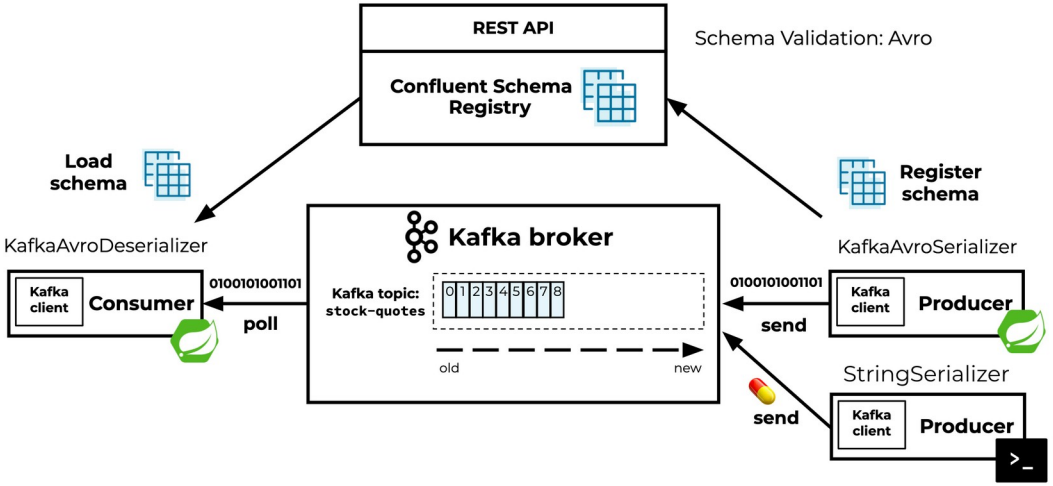


Image Credit: https://www.confluent.io/blog/spring-kafka-can-your-kafka-consumers-handle-a-poison-pill/

© 2026 by Innovation In Software Corporation

The Poll Loop

- The container drives execution
  - From the consumer's code perspective, that loop is invisible
  - The container plays the role of the event dispatcher:
    - Kafka delivers events
    - The container deserializes them
    - The container invokes the appropriate handler
  - The application code never sees the loop, only the events

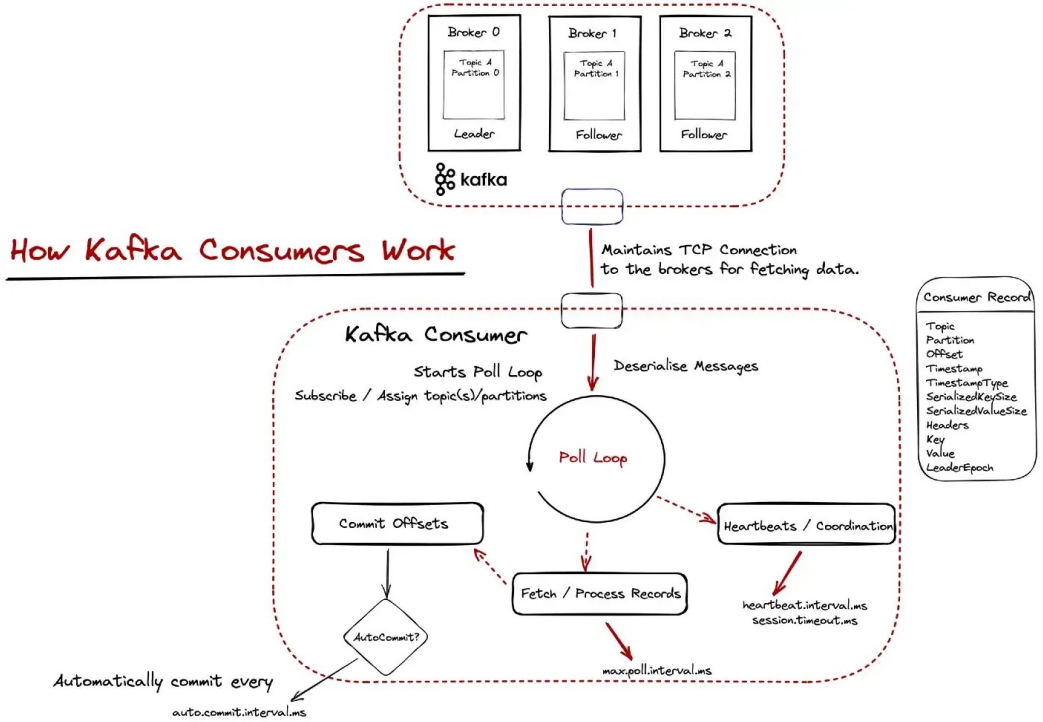


Image Credit: https://rockthejvm.com/articles/kafka-101-clients

© 2026 by Innovation In Software Corporation

Coding

- Writing handlers
  - Your method is the reaction, not the caller
    - Don't try to "drive" the consumer from your code
    - Trust the container to deliver events when they arrive
  - Processing time matters globally, not just locally
    - A slow handler delays everything behind it on its partition
    - Affects consumer lag, group rebalancing, system responsiveness
  - Errors are caught by the container, not by you
    - Throw exceptions; let the configured error handler take over
    - Don't try to swallow errors in business logic — that defeats DLT routing
  - Idempotency is the handler's responsibility
    - The container may redeliver records on failure or rebalance
    - Handlers should be safe to invoke multiple times for the same event

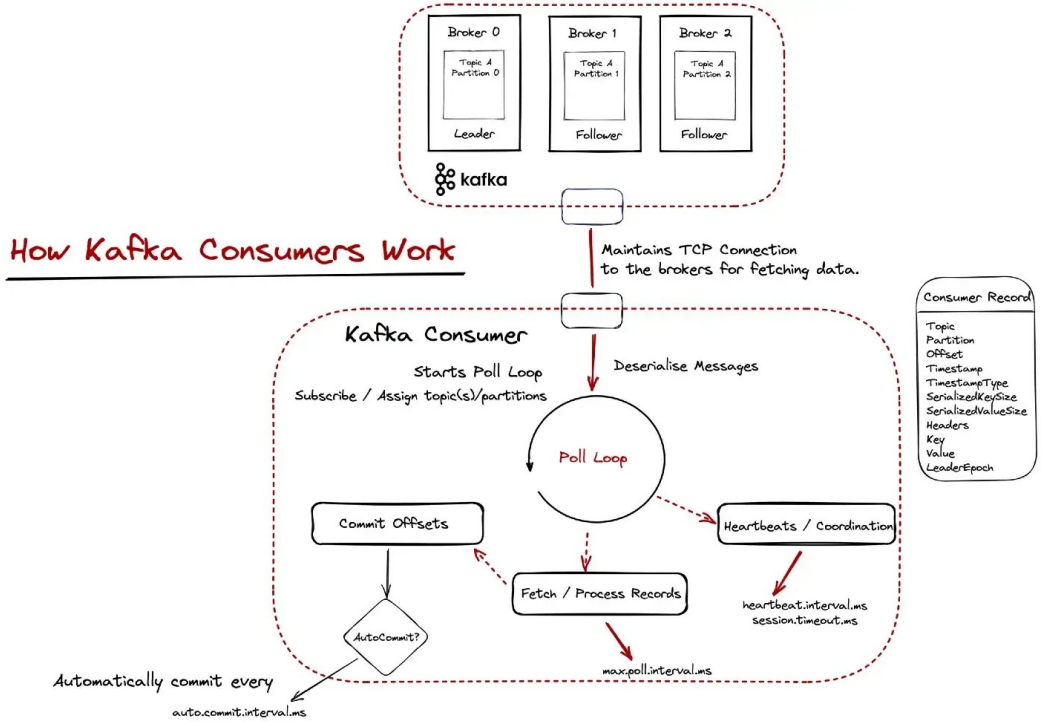


Image Credit: https://rockthejvm.com/articles/kafka-101-clients

© 2026 by Innovation In Software Corporation

Idempotency

- Kafka guarantees at-least-once delivery by default
  - A handler may be called multiple times for the same event
    - Container crashed before committing the offset
    - Rebalance reassigned the partition mid-batch
    - Application restarted before commit
  - An idempotent handler produces the same result whether called once or N times
  - Common patterns:
    - Check before insert (does this order already exist?)
    - Use natural keys instead of generated ones
    - Track processed event IDs in a deduplication store

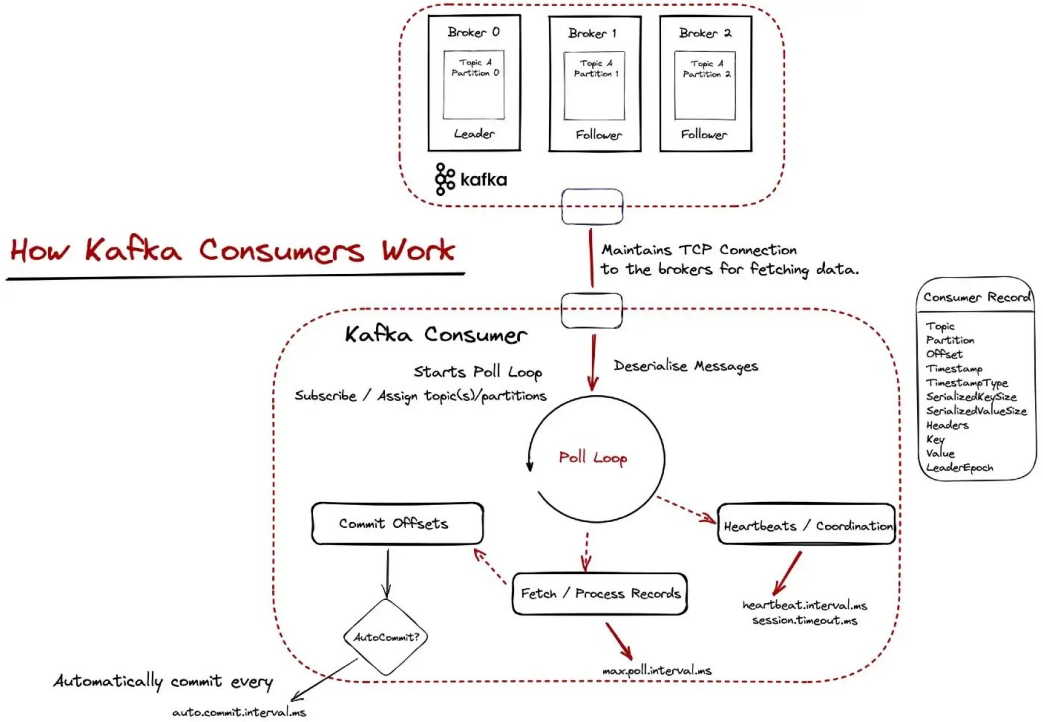
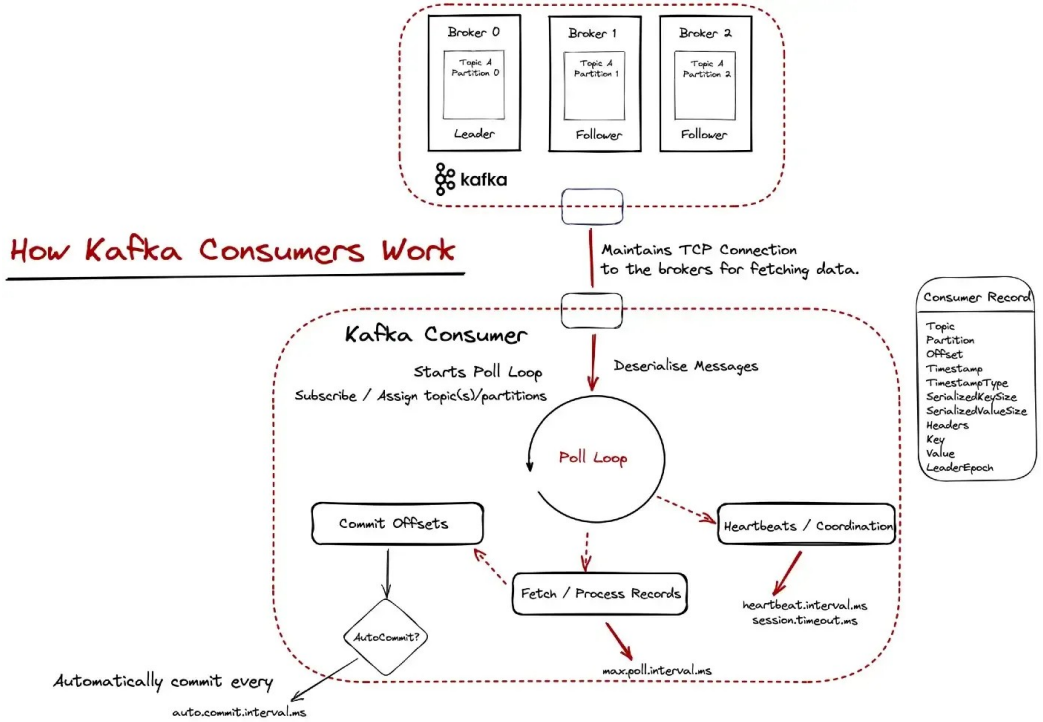


Image Credit: https://rockthejvm.com/articles/kafka-101-clients

© 2026 by Innovation In Software Corporation

Coding

- Each consumer is one reaction
  - One consumer = one team's reaction to one kind of event
  - Each handler is the code expression of an architectural decision:
    - "When orders are placed, our service updates inventory"
    - "When payments arrive, our service sends a confirmation"
    - "When users sign up, our service starts onboarding"
  - Adding a new handler = adding a new architectural reaction



© 2026 by Innovation In Software Corporation

Image Credit: https://rockthejvm.com/articles/kafka-101-clients

Honor the Model

- Code discipline required to leverage architecture
  - Loose coupling at the architecture level requires
    - Handlers don't reach back to producers for clarification
    - Handlers don't assume specific producer behavior beyond the event contract
    - Handlers don't depend on specific timing or ordering across topics
  - Resilience at the architecture level requires:
    - Handlers handle their own errors gracefully
    - Handlers are idempotent
    - Handlers don't propagate failures to other consumers
  - The architecture's benefits are not automatic
    - They emerge from disciplined code

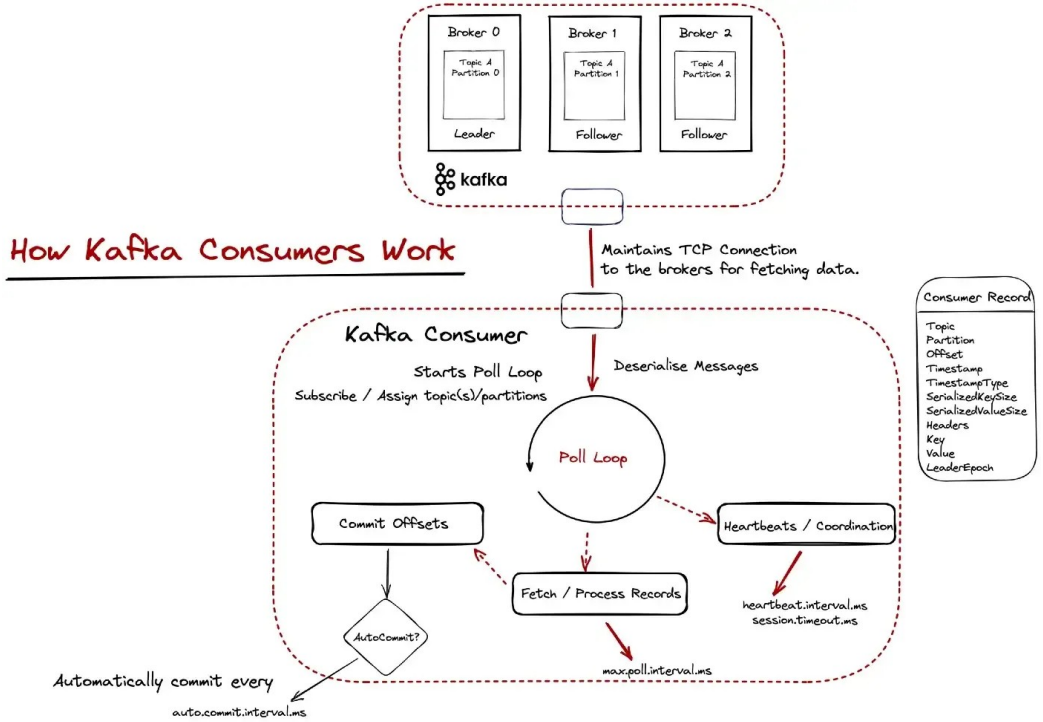


Image Credit: https://rockthejvm.com/articles/kafka-101-clients

© 2026 by Innovation In Software Corporation

WebClient

© 2026 by Innovation In Software Corporation

WebClient

- The Async HTTP Client
  - Spring Boot has two HTTP client options
    - RestTemplate: synchronous, blocking, older
    - WebClient: asynchronous, non-blocking, current
  - WebClient is the reactive counterpart to RestTemplate
    - Even in mostly-synchronous applications, WebClient is the recommended choice
  - Understanding WebClient = applying event-driven programming to HTTP calls

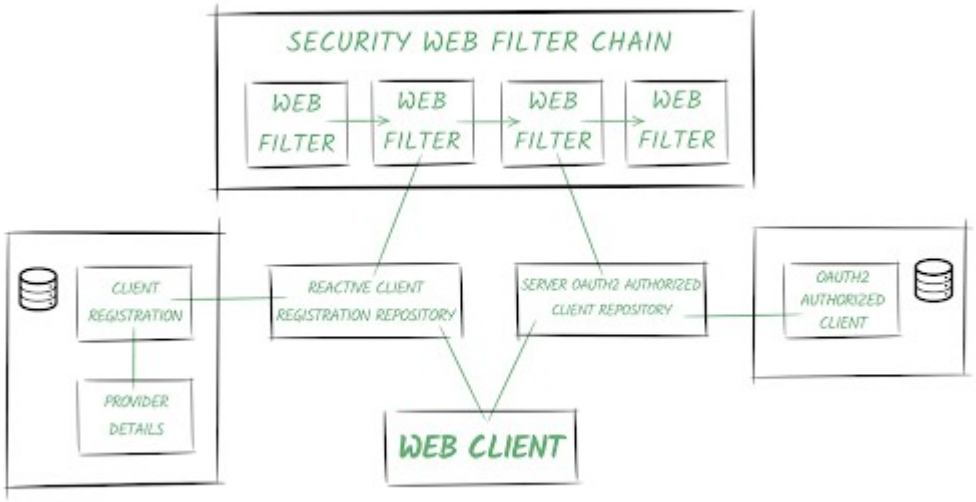


Image Credit: https://www.java67.com/2022/12/10-examples-of-spring-5-webclient-in.html

© 2026 by Innovation In Software Corporation

RestTemplate Comparison

- RestTemplate blocks the calling thread until the response arrives
- WebClient returns immediately
  - Processing happens in a callback when the response arrives

```java
// RestTemplate — synchronous, blocking
ResponseEntity<User> response = restTemplate.getForEntity(
    "https://api.example.com/users/" + id, User.class);
User user = response.getBody();
process(user);   // runs after the call returns

// WebClient — asynchronous, non-blocking
webClient.get()
    .uri("https://api.example.com/users/" + id)
    .retrieve()
    .bodyToMono(User.class)
    .subscribe(user -> process(user));   // runs when the response arrives
```

© 2026 by Innovation In Software Corporation

Reactive Operators

- Transforming future values
  - map
    - Ttransform the value when it arrives (synchronously)
  - flatMap
    - Chain another async operation that itself returns a Mono
  - filter
    - Drop the value if it doesn't match a predicate
  - onErrorResume
    - Handle errors with a fallback
  - These compose into expressive, declarative pipelines

```
Mono<UserSummary> summary = webClient.get()
    .uri("/users/" + id)
    .retrieve()
    .bodyToMono(User.class)
    .map(user -> user.normalizeName())          // synchronous transform
    .flatMap(user -> fetchOrders(user))         // chain another async call
    .map(orders -> summarize(orders));          // another sync transform
```

© 2026 by Innovation In Software Corporation

WebClient Example

- Putting it together
  - WebClient is injected via the auto-configured WebClient.Builder
  - The method returns a Mono
    - The caller decides when to subscribe
  - The HTTP call hasn't happened yet when the method returns
    - In otherwords, the thread isn’t blocking waiting for an event

```java
@Service
public class UserEnrichmentService {

    private final WebClient webClient;

    public UserEnrichmentService(WebClient.Builder builder) {
        this.webClient = builder.baseUrl("https://api.example.com").build();
    }

    public Mono<EnrichedOrder> enrich(Order order) {
        return webClient.get()
            .uri("/users/{id}", order.userId())
            .retrieve()
            .bodyToMono(User.class)
            .map(user -> new EnrichedOrder(order, user));
    }
}
```

© 2026 by Innovation In Software Corporation

WebClient Example

- Nothing happens until you subscribe
  - The subscription step is where deferred reactive code becomes actual execution
  - Most commonly, in Spring WebFlux controllers
    - Returning a Mono from a controller method causes Spring to subscribe automatically as part of the response handling
    - The framework knows how to drive a Mono to completion and turn the result into an HTTP response
  - For developers new to reactive programming
    - The most common bug is forgetting that subscription is required
    - They construct an elaborate reactive chain, and nothing happens because nothing subscribed

```java
// This DOES NOT make any HTTP request:
Mono<User> userMono = webClient.get().uri("/users/1").retrieve().bodyToMono(User.class);

// This DOES make the request:
userMono.subscribe(user -> log.info("Got user: {}", user));

// This ALSO makes the request, by blocking until completion:
User user = userMono.block();   // converts back to synchronous

// In a Spring controller, returning a Mono triggers subscription automatically:
@GetMapping("/users/{id}")
public Mono<User> getUser(@PathVariable Long id) {
    return webClient.get().uri("/users/" + id).retrieve().bodyToMono(User.class);
}
```

© 2026 by Innovation In Software Corporation

WebClient Example

- Async all the way down
  - The handler kicks off an async chain
    - WebClient call → Kafka send → completion
  - The handler thread is not blocked during any of it
  - Errors anywhere in the chain are caught at the end

```java
@KafkaListener(topics = "orders.placed")
public void handleOrder(OrderPlacedEvent event) {
    enrichmentService.enrich(event)             // returns Mono<EnrichedOrder>
        .flatMap(enriched ->
            kafkaTemplate.send("orders.enriched",
                enriched.orderId(),
                enriched).thenReturn(enriched))   // chain the Kafka send
        .subscribe(
            enriched -> log.info("Processed: {}", enriched.orderId()),
            error -> log.error("Failed", error)
        );
}
```

© 2026 by Innovation In Software Corporation

WebClient Example

- Async all the way down
  - A Kafka event arrives, triggering the handler
    - The handler calls enrichmentService.enrich(event), which returns a Mono as an async future result
  - The handler chains a Kafka send onto the completion of the enrichment, also async
    - Then it subscribes, providing success and error callbacks
  - Nowhere does the handler thread block
  - The handler returns almost immediately, having set up the async chain.
    - The actual enrichment HTTP call and the subsequent Kafka send happen on other threads
    - One handler thread can set up many such chains in rapid succession
    - This is the high-throughput, low-resource async model in action

```java
@KafkaListener(topics = "orders.placed")
public void handleOrder(OrderPlacedEvent event) {
    enrichmentService.enrich(event)             // returns Mono<EnrichedOrder>
        .flatMap(enriched ->
            kafkaTemplate.send("orders.enriched",
                enriched.orderId(),
                enriched).thenReturn(enriched))   // chain the Kafka send
        .subscribe(
            enriched -> log.info("Processed: {}", enriched.orderId()),
            error -> log.error("Failed", error)
        );
}
```

© 2026 by Innovation In Software Corporation

Integration Points

- Practical integration points
  - Enrichment
    - Consume an event, fetch additional data via WebClient, produce enriched event
  - Validation
    - Consume an event, call a validation API, accept or route to DLT
  - External integration
    - Consume an event, call a partner API as the side effect
  - Synchronous reads
    - Spring controllers using WebClient to call other services for queries
  - These are everyday patterns in event-driven microservices

```java
@KafkaListener(topics = "orders.placed")
public void handleOrder(OrderPlacedEvent event) {
    enrichmentService.enrich(event)             // returns Mono<EnrichedOrder>
        .flatMap(enriched ->
            kafkaTemplate.send("orders.enriched",
                enriched.orderId(),
                enriched).thenReturn(enriched))   // chain the Kafka send
        .subscribe(
            enriched -> log.info("Processed: {}", enriched.orderId()),
            error -> log.error("Failed", error)
        );
}
```

© 2026 by Innovation In Software Corporation

Questions?